

A Graph Reinforcement Learning Framework for Routing in a Hybrid Fiber and Free-Space QKD Network

Author Names Withheld for Draft

Abstract—Hybrid quantum key distribution (QKD) routing must account for link feasibility and changing channel conditions. We present a simulator and graph neural network (GNN) policy trained with proximal policy optimization (PPO) for next-hop routing over a modeled hybrid fiber and free-space optical network. In a paired matrix of 108 episodes spanning six ordered endpoint pairs and six season/time conditions, one trained GNN checkpoint completed 99 episodes (91.7%; Wilson 95% confidence interval, 84.9–95.6%). It substantially exceeded distance-weighted Dijkstra (33/108), random routing (1/108), and greedy maximum-secret-key-rate routing (0/108), with no invalid action proposals or environment fallbacks. However, breadth-first search (BFS) completed all 108 episodes and used fewer hops; paired analysis favored BFS in success and route length. On 20 generated seven-node static-channel cases, the GNN matched the exhaustive reward optimum in 11/18 successful policy episodes. These results show that the agent can exploit the simulator’s feasibility mask and state-dependent reward more effectively than several link-cost heuristics, while not demonstrating an advantage over BFS. All rates are normalized proxies, and all results are simulator-derived rather than field or security validation.

Index Terms—Quantum key distribution, trusted-node networks, routing, graph neural networks, attention, proximal policy optimization, free-space optical communication.

1. INTRODUCTION

Quantum key distribution (QKD) supplies shared secret key material using quantum communication and authenticated classical post-processing. Interest in QKD has grown alongside the prospect of quantum algorithms that threaten some public-key systems [1]. In a trusted-node network, intermediate nodes relay key material between distant endpoints. The resulting network routing problem must account for link quality and the availability of stored key material, in addition to conventional path length and hop count [2]–[6]. The usable rate is constrained by the weakest link on a relayed path, while changing channel conditions can make a route that was feasible at one time infeasible later.

Prior work has compared routing policies for meshed trusted-node networks under traffic and key-store models, including methods with different levels of information available to the controller [4]. Separately, finite-key analysis for simplified trusted-node (STN) chains establishes that security and computational cost depend on protocol details and block size [7]. These two lines of work motivate a routing study that models link uncertainty and preserves a clear boundary

between network-level route selection and cryptographic security proofs.

This paper describes an extensible simulator and a graph-based routing agent for a hybrid fiber and free-space optical (FSO) QKD network. We represent routing as a partially observed sequential decision problem. An agent observes current route state and link features, selects a feasible adjacent node, and receives reward shaped by progress, link utility, security constraints, and route costs. A graph neural network (GNN) encodes node and edge context, while proximal policy optimization (PPO) trains the policy from simulator interaction [8]–[10]. The implementation includes a hard action mask for invalid QBER/SKR links and supports an optional attention based candidate-scoring component.

The current contribution is methodological and software oriented. It includes: 1) a road-corridor proxy topology for seven Indian city hubs, with candidate FSO bypasses; 2) a layered fiber and FSO link simulator with time-varying, temporally correlated FSO turbulence; 3) a configurable asymptotic secret-key-rate proxy and an opt-in vacuum-plus-weak-decoy finite-key engineering estimator; 4) a masked graph-policy training and evaluation workflow; and 5) an evidence ledger and validation protocol that separate derived simulation outputs from measurements. The present results are limited to archived simulator experiments. They do not establish deployed-network performance, composable finite-key security, or field availability.

The paper is organized as follows. Section II formulates the routing setting and summarizes the QKD model. Section III describes the simulator and evaluation setup. Section IV presents the learned routing policy and comparator formulations. Section V reports paired route-level results and a small-graph oracle analysis. Section VI discusses the limits of the evidence and the requirements for a network-level study. Section VII concludes the paper.

2. PROBLEM FORMULATION AND QKD MODEL

A. Basic Principles of QKD

QKD protocols combine a quantum channel with authenticated classical communication. BB84 uses nonorthogonal signal states to allow the communicating parties to estimate disturbance and distill key material [11]. Practical transmitters commonly use weak coherent pulses rather than ideal single-photon sources. Decoy-state methods use pulses with different

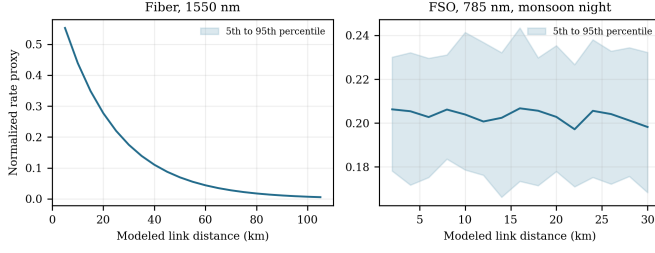


Fig. 1. Modeled normalized rate proxy versus link distance for the configured fiber and FSO scenarios. Shading shows sampled scenario variation, not measured uncertainty or validation against a published curve.

intensities to estimate single-photon contributions and counter photon-number-splitting attacks [12], [13].

Finite-key operation must include statistical fluctuations, error-correction leakage, and security parameters. Asymptotic rate formulas are useful for link-level screening, but they do not by themselves certify a finite block. Entropic uncertainty and composable security analyses provide established foundations for finite-size bounds [14]–[16]. For STN chains, the parties and parity announcements couple information across links. A finite-key bound for such a chain is therefore protocol-specific and cannot be replaced by summing independent per-link asymptotic rates [7].

B. Generation and Relaying of Secret Keys

In a conventional trusted-node chain, neighboring QKD modules establish link keys. The network forwards end-to-end key material through trusted relays, commonly using one-time-pad operations. For a path with link key rates R_i , the instantaneous end-to-end service rate is limited by the slowest required resource, and practical delivery also depends on stored key inventory and demand. Key pools therefore couple routing decisions over time [4], [17].

The simulator’s default link utility is a normalized rate proxy, not a throughput in bits per second. A fiber channel is attenuated as a function of distance, and a detector model contributes efficiency and dark counts. The code computes a QBER proxy and a rate proxy from these quantities. The optional finite-key BB84 mode adds decoy-state and uncertainty terms, but has not been independently verified as a composable proof against measured detection counts. Fig. 1 illustrates the configured distance-dependent proxy under selected channel scenarios.

In a conventional trusted-node (TN) chain, each neighboring pair performs QKD and the intermediate TNs relay link keys. Each TN is trusted with the key material it handles. In a simplified trusted-node (STN) chain, intermediate devices can perform state preparation or measurement and announce parities of raw link data, while error correction and privacy amplification are performed by the end users. The security analysis therefore treats the whole chain and its correlated public announcements, rather than assigning an independent single-link proof to every hop. Krawec *et al.* derive a finite-key bound for this STN setting and analyze its cost trade-off against conventional TN chains [7]. In this project, that proof is background motivation only: the implemented estimator is

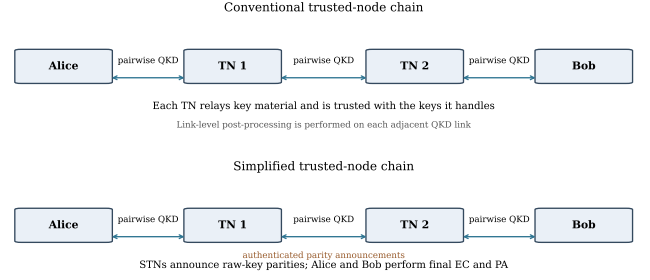


Fig. 2. Conceptual key-relay roles in conventional TN and STN chains. The STN security proof is cited background and is not implemented by the simulator.

not their chain proof, and the simulated relay types do not establish a composable security guarantee.

C. Application of QKD to Meshed Networks

Long fiber spans motivate intermediate trusted nodes. FSO links can provide candidate bypasses where a direct fiber span is unavailable or degraded. Their channel quality depends on atmospheric attenuation, turbulence, pointing, and availability. Terrestrial FSO design methods identify these as separate budget terms and require local environmental information for credible planning [18], [19]. The simulator models time-of-day and seasonal scenario variables, but its current FSO distributions are assumptions rather than fits to site-specific Indian weather traces.

D. State of the Art in QKD Routing

Classical shortest-path methods provide reproducible baselines but typically optimize a fixed edge cost [20]. QKD network studies also consider key-store utilization, request blocking, load balancing, and management overhead [3], [4]. Recent learning-based work has applied PPO to entanglement routing in quantum communication networks and state-dependent routing to networks combining trusted nodes and repeaters [21], [22]. These settings differ from trusted-node key-pool routing with hybrid fiber and FSO links, so their results do not validate the present simulator. In dynamic settings, sequential decision methods can account for state-dependent link availability, with dynamic programming providing a classical foundation for sequential optimization [23]. PPO is a clipped policy-gradient method that supports repeated minibatch updates from sampled rollouts [8]; generalized advantage estimation reduces variance in return estimates [24]. The partially observed formulation follows standard POMDP and reinforcement-learning foundations [25], [26].

GNNs represent relationships among nodes and edges through message passing. Graph convolution provides a compact aggregation rule [9]; inductive neighborhood aggregation and general message-passing formulations provide related design patterns [27], [28]. Attention mechanisms allow learned weighting of neighborhood information [10], [29]. These properties are relevant when one policy must operate on a network graph and select among adjacent actions. They do not

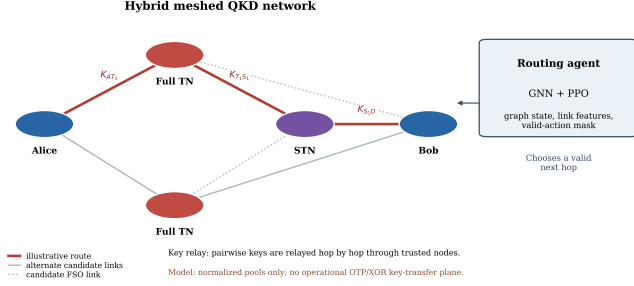


Fig. 3. Hybrid meshed QKD routing and key-relay schematic. Red links show an illustrative multihop route with per-link key labels; gray links show alternatives. The controller represents the centralized graph observation used in simulation. Key transfer and the OTP/XOR relay plane remain conceptual abstractions.

guarantee that the agent learns a physically appropriate link preference, which must be evaluated with controlled tests.

3. SIMULATOR ARCHITECTURE AND EXPERIMENTAL SETUP

A. Architecture

The simulator has four logical components: topology and link profiles, a time-stepped environment, a routing policy, and an evaluation layer. The topology represents QKD-capable sites and candidate communication links. The environment samples channel conditions, enforces link security and reachability constraints, tracks route history and key-pool state, and returns an observation and reward. The policy selects a next-hop action. Evaluation replays fixed seeds and compares learned policies with conventional baselines. The present software does not implement a production key-management plane or a QKD device control interface.

B. Security Requirements on the Network Controller

The reference architecture assumes a controller with topology and link-health information. This manuscript distinguishes a local policy that acts on its current observation from a centralized oracle that can inspect a fixed simulated network state. In a deployed QKD network, telemetry on key-pool levels and link health would need authenticated and integrity-protected control channels. The current simulator does not implement a network controller protocol, attacks on telemetry, or a control-message counter. Thus, management overhead and telemetry security are architectural requirements, not measured outcomes here.

C. Simulation Setup

1. Topology and Corridor Data

The current topology uses seven city hubs: Delhi, Jaipur, Mumbai, Bangalore, Hyderabad, Kolkata, and Chennai. OpenStreetMap driving routes provide candidate fiber-corridor geometry. These routes are planning proxies, not verified operator fiber routes, surveyed alignments, or guaranteed rights-of-way. The graph contains 2,370 modeled nodes and 2,653 links, including 273 fiber links and 2,380 candidate FSO links. FSO line of sight, terrain clearance, and access to physical infrastructure are not verified. Topology geometry is archived

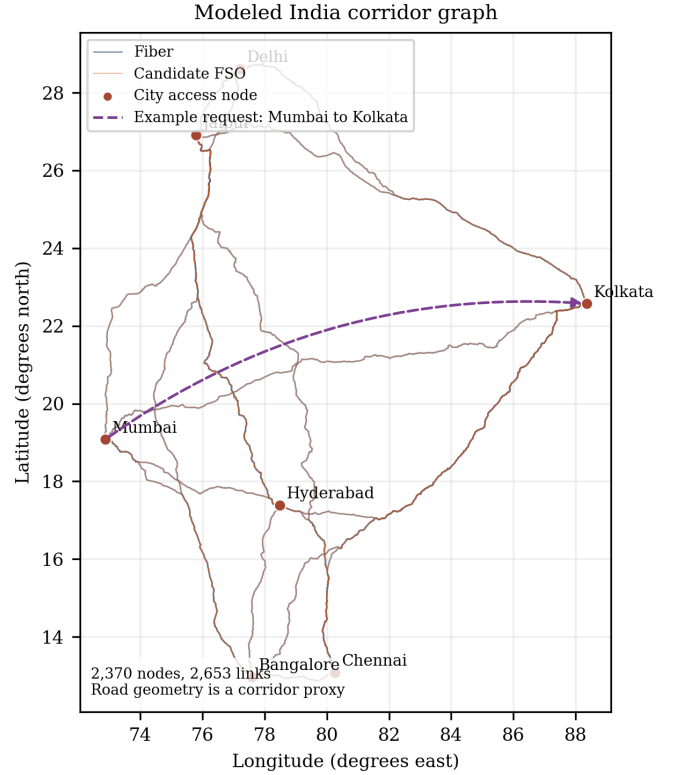


Fig. 4. Modeled corridor graph with fiber links and candidate FSO links. The dashed Mumbai-to-Kolkata line marks an example request, not a physical link. Road geometry is a corridor proxy; FSO line of sight is assumed.

TABLE I
REPRESENTATIVE LINK-MODEL PARAMETERS AND EVIDENCE CLASS

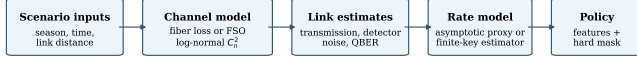
Quantity	Configured value	Evidence class
Fiber wavelength	1550 nm	device profile
Fiber attenuation	0.20 dB/km	inherited target
Fiber detector efficiency	0.93	measured device value
Fiber dark-count bound	$< 70 \text{ s}^{-1}$	measured upper bound
FSO wavelength	785 nm	scenario assumption
FSO detector efficiency	0.60	scenario assumption
FSO dark counts	100 s^{-1}	scenario assumption
FSO temporal correlation	0.85	uncalibrated assumption

with the research data. OpenStreetMap data use is governed by its applicable attribution and database terms [30].

2. Link and Noise Models

Fiber loss is represented using wavelength-dependent attenuation and excess loss. Detector efficiency, dark-count bounds, and a visibility assumption contribute to detection and QBER proxies. The selected fiber device profile is based on published/vendor-reported ID Quantique ID281 values at 1550 nm. The FSO channel is a separate assumed 785-nm scenario with its own efficiency and dark-count settings. These are not a single matched commercial optical terminal profile.

FSO turbulence is represented with a correlated latent log- C_n^2 process. The environment samples availability and channel conditions by time step; FSO corridor outages are held fixed for the episode to avoid a bypass disappearing between its relay segments. These choices create repeatable stress sce-



Current FSO model: correlated log-normal turbulence, atmospheric and pointing terms, background and detector noise

No Gamma-Gamma irradiance model or generalized amplitude-damping channel is implemented

Fig. 5. Implemented channel-to-routing coupling. The figure distinguishes the current log-normal scenario model from other turbulence and quantum-channel models that are not implemented.

narios. They are not observations of weather on the modeled routes. A detailed parameter ledger and provenance categories are supplied in `PHYSICS_EVIDENCE.md`.

The implemented FSO model uses a correlated log-normal turbulence process with atmospheric, diffraction, pointing, and background terms. It does not implement a Gamma-Gamma irradiance model or a generalized amplitude-damping channel. Detector and channel noise enter through the project’s QBER and rate calculations. Fig. 5 summarizes this implemented coupling; the optional finite-key estimator remains a simplified engineering model.

The default rate mode is an asymptotic, normalized proxy of the form

$$r_{\text{proxy}} = p_{\text{det}} [1 - 2h_2(Q)], \quad (1)$$

where p_{det} is modeled detection probability, Q is modeled QBER, and h_2 is binary entropy. The expression is used only as a simulator utility. It is not a composable key rate, does not include all protocol terms, and is not reported in bits per second. An opt-in decoy-state mode estimates finite-key contributions and uncertainty, but that mode is also not a formal security certificate.

3. Environment and Protected Conditions

At each time step the observation contains the current node, destination-relative distance, node type, normalized degree, candidate link features, and route history features. Absolute coordinates and node identifiers are excluded from the configured relative feature mode. Candidate actions are limited to adjacent nodes. Links that violate the hard QBER threshold or have invalid rate are masked before action sampling. The threshold is configured at $Q_{\text{max}} = 0.11$.

The reward includes destination progress, distance-aware link utility, a step cost, revisit and link-switch penalties, a backtracking penalty, and explicit security and key-pool depletion penalties. The link utility is prorated by modeled link distance so subdividing a span does not multiply its total rate reward. The experimental protocol records these as protected or adjustable values and stores resolved configurations with each run.

The observation contains five node features and nine features for each candidate edge. For the archived run, node inputs use destination-relative distance and normalized degree, while edge inputs use the simulator quantities listed in Table II. The defence reward profile uses weights $w_{\text{progress}} = 0.2$, $w_{\text{margin}} = 0.8$, $w_{\text{SKR}} = 1.2$, $w_{\text{pool}} = 0.4$, $w_{\text{latency}} = 0.4$,

TABLE II
OBSERVATION FEATURES FOR EACH CURRENT NODE AND CANDIDATE EDGE

Feature group	Components
Node (5)	City indicator; full-TN indicator; STN indicator; destination-relative distance; normalized degree.
Edge (9)	QBER; normalized secret-key-rate proxy; FSO indicator; normalized distance; projected chain error; normalized key-pool state; link-type switch; visited-node flag; destination-distance progress.
Action mask	Edge is adjacent, passes the edge QBER threshold, has positive modeled rate, and remains below the projected chain-error threshold.

TABLE III
GNN-PPO CHECKPOINT AND PAIRED TRANSFER-EVALUATION PROTOCOL

Setting	Value
PPO training endpoints	Delhi to Chennai
Training budget	15 behavior-cloning and 50 PPO epochs
Behavior-cloning endpoints	Randomized
Hidden dimension	64
Transfer pairs	Six ordered pairs excluding Delhi–Chennai
Weather/time conditions	Six selected season and hour settings
Paired seeds	Three per pair-condition (108 episodes per policy)
QBER hard threshold	0.11
Fiber outage probability	0.01
Episode limit	144 steps

$w_{\text{energy}} = 0.4$, $w_{\text{congestion}} = 0.5$, and destination bonus 4.0. Additional configured penalties are -0.05 per step, -0.6 for a revisit, -0.2 for switching link type, -0.5 for backtracking, -30 for a security breach, and -60 for pool depletion.

4. Training and Evaluation Setup

The GNN policy uses three message-passing layers, a shared actor-critic trunk, and a masked action head. The implementation uses a monotone log-rate prior and an explicit distance-progress prior. Behavior cloning can initialize the policy from shortest-path demonstrations before PPO fine-tuning. PPO uses clipped policy updates, generalized advantage estimation, reward normalization, gradient clipping, and checkpointed normalization state. The optional attention model is implemented in `qkd_attention.py`; the GNN-PPO runs summarized below use the graph policy configuration specified in their saved YAML files.

Baselines include random feasible action selection, shortest distance with Dijkstra, breadth-first hop count, and maximum instantaneous rate. Baseline outcomes are sensitive to whether they observe current rates, how they break ties, and how repeated states are handled. Each comparison must therefore use a common environment, endpoint pair, seed set, termination rule, and action feasibility mask. The packaged experiment protocol identifies the controlled conditions and required diagnostics.

4. ROUTING POLICY AND LEARNING OBJECTIVE

A. Hop-Count Comparator

The hop-count comparator uses breadth-first search over the modeled topology. Prior trusted-node routing studies evaluate

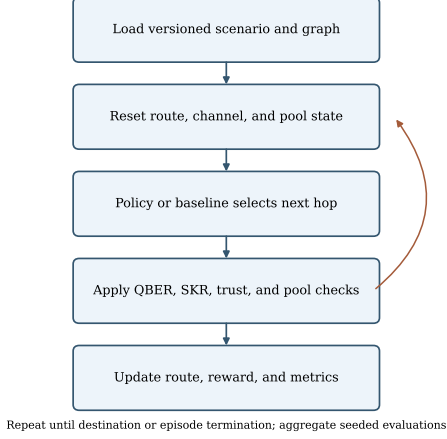


Fig. 6. Simulation and evaluation flow used by the routing environment. The diagram describes the implemented single-route experiment loop.

multiple loopless k -shortest paths and check key-store feasibility on each candidate. The present baseline selects a next hop and does not implement that k -candidate buffer-feasibility procedure. It is a topology-routing comparator rather than a demand-level key-pool algorithm.

B. Perfect-Information Comparator

An oracle comparator can use future demand or channel-state information unavailable to an online policy. The present simulator has neither a demand matrix nor a future-state evaluation interface, so no perfect-information result is reported.

C. Graph and Sequence Policy Encoders

The repository provides both an `LSTMActorCritic` and a `GNN-PPO` policy. The LSTM is a graph-blind encoder over an ordered node-feature array, not a recurrent demand-history predictor. The GNN-PPO policy operates on the current graph and link observation. The reported experiment uses GNN-PPO; no matched LSTM comparison is included.

1. Policy Formulation and PPO Training

Let the network be a graph $G = (V, E)$, with source s , destination d , and time-varying state x_t . The agent observes o_t and selects an adjacent node action a_t from the feasible set $\mathcal{A}(o_t)$. Transition dynamics sample channel conditions and update route and key-pool state. Because future turbulence and outages are not exposed to the policy, the routing task is partially observed.

The objective is to find a policy $\pi_\theta(a_t | o_t)$ that maximizes expected discounted return

$$J(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^T \gamma^t r_t \right], \quad (2)$$

subject to hard feasibility constraints on link adjacency, QBER, and modeled key availability. The reward is a training signal; reported evaluation metrics include route success, successful-route hop count, revisits, QBER/rate validity, and seed variation. A reward value alone is not interpreted as network service quality.

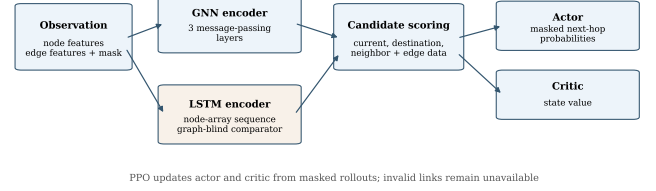


Fig. 7. GNN-PPO policy architecture and the available graph-blind LSTM encoder comparator. The LSTM input is an ordered node-feature array, not a time-series demand forecast.

2. Graph Policy and Action Mask

For each node v , the message-passing stack computes an embedding h_v from the node state and valid neighboring features. The actor produces logits $z_{v,u}$ for adjacent candidate actions. Invalid candidates are assigned negative infinity before softmax:

$$\pi_\theta(u | o_t) = \text{softmax}_{u \in \mathcal{A}(o_t)}(z_{v_t,u}). \quad (3)$$

Invalid links contribute no message in either direction. This separates the safety constraint from the learned preference: training cannot select a currently invalid edge, while model quality determines which of the remaining edges is chosen.

In the optional attention variant, candidate embeddings are combined using learned compatibility scores, following neighborhood attention formulations [10]. Attention is treated as an architectural choice. Its value must be established by an ablation with the same training budget and seed distribution. The current results do not isolate a causal benefit from attention.

3. PPO Training

The policy is updated by maximizing the clipped surrogate objective

$$L^{\text{clip}}(\theta) = \mathbb{E}_t \left[\min \left(\rho_t(\theta) \hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right], \quad (4)$$

where $\rho_t(\theta)$ is the new-to-old action probability ratio and $\hat{A}_t$ is the estimated advantage. PPO is combined with a value loss and entropy regularization. Hyperparameters are resolved from a versioned YAML file and included with the experiment artifact. The settings for the transfer-evaluation checkpoint and paired episode seeds are archived in the supplementary package.

D. Centralized Comparator Formulations

A centralized link-weight comparator can assign edge costs from link state and key-pool information, while an integer linear program can optimize simultaneous demands subject to link capacities. The current single-route simulator does not provide the demand matrix or calibrated key generation and consumption units needed to evaluate either method on a shared objective. These comparator families are outside the measured experiment reported here.

1. Link-Weight Optimization

A published trusted-node routing heuristic sets an edge weight from key-store emptiness E_e as $W_e = \exp(aE_e)$, with tuning parameter a [4]. A rate-aware extension could include an inverse-rate term, but that would define a different method. The present simulator has no demand stream or multi-demand key-store dynamics, so neither heuristic is included in the measured evaluation.

2. Integer Linear Programming

For completeness, a demand-level oracle can be written using binary variable x_{kp} , which is one when request k is assigned candidate path p , request demand b_k , and edge key availability C_e . The standard capacity-constrained formulation is

$$\max_{x_{kp}} \sum_{k \in \mathcal{K}} \sum_{p \in \mathcal{P}_k} x_{kp}, \quad (5)$$

$$\text{s.t.} \quad \sum_{p \in \mathcal{P}_k} x_{kp} \leq 1, \quad k \in \mathcal{K}, \quad (6)$$

$$\sum_{k \in \mathcal{K}} \sum_{p \in \mathcal{P}_k: e \in p} b_k x_{kp} \leq C_e, \quad e \in \mathcal{E}, \quad (7)$$

$$x_{kp} \in \{0, 1\}. \quad (8)$$

The current environment processes one route episode and lacks a time-indexed request matrix and calibrated key-store refill process. This ILP is therefore a mathematical comparator definition, not a measured result.

5. RESULTS

A. Paired Endpoint and Season/Time Evaluation

We evaluated one fixed GNN-PPO checkpoint on six ordered endpoint pairs, six season/time conditions, and three shared environment seeds per pair-condition combination. The pairs were Delhi–Mumbai, Mumbai–Chennai, Kolkata–Delhi, Bangalore–Hyderabad, Jaipur–Kolkata, and Hyderabad–Mumbai; the PPO fixed training pair, Delhi–Chennai, was excluded. Conditions covered normal, summer, winter, and monsoon scenarios at selected day and night hours. Five policies received the same episode seeds, yielding 108 paired episodes per policy. The GNN checkpoint was trained with PPO on Delhi–Chennai, but behavior cloning randomized endpoints; the evaluation is therefore a cross-pair transfer diagnostic, not a strict end-to-end endpoint holdout. Success-rate intervals in Table IV are Wilson score intervals across episodes for this one checkpoint, not intervals across independently trained models.

The strongest result for the learned agent is its performance against the distance-weighted and greedy link-quality comparators: it completed 99 episodes, compared with 33 for Dijkstra-km, one for Random, and none for Max-SKR. It proposed no invalid actions and triggered no environment fallbacks. This indicates that the GNN policy can combine route context and the hard feasibility mask more effectively than these fixed local cost rules in the tested simulator conditions. It is not a general routing advantage: BFS-hop succeeded in every paired episode and produced substantially shorter successful routes.

The GNN’s completion rate varied by tested condition. It succeeded in all 18 episodes for each of winter at 22:00 and monsoon at both 02:00 and 22:00. Success was 16/18 in summer at 22:00, 15/18 in normal conditions at 22:00, and 14/18 in normal conditions at 02:00. These small condition-specific samples suggest the strongest observed performance occurred in the tested winter and monsoon cases; they do not establish general weather robustness. BFS remained 18/18 in every condition.

The no-valid-action column counts terminal states in which a policy had no feasible next hop; these are not invalid proposals. The GNN, BFS, and Dijkstra policies reached no such states in this matrix, whereas random and Max-SKR did so 42 and 10 times, respectively.

The paired outcomes sharpen this comparison. BFS had nine successes on episodes where GNN-PPO failed; there were no episodes with the opposite outcome. An exact two-sided McNemar test gives $p = 0.0039$. Among the 99 episodes where both policies succeeded, the GNN used 16.00 more hops on average (median difference, six hops). A bootstrap that resampled the six ordered endpoint-pair clusters gave a 95% interval of 6.48–25.97 additional GNN hops. Because that interval uses only six clusters, it is coarse; it nevertheless agrees with the descriptive comparison and rules out a claim that the GNN is the best route-completion or hop-minimization policy in this matrix. Episode-level Wilson intervals do not represent training-seed uncertainty.

B. Exact Reward Oracle on Small Graphs

To test policy behavior against a tractable optimum, we generated five connected seven-node graphs and evaluated four channel seeds per graph. Link states were frozen within each episode, with a 5% per-edge fiber outage probability. An exhaustive enumeration of feasible simple paths produced a reward oracle for all 20 cases. The GNN succeeded in 18/20 cases (90%) and exactly matched the oracle reward in 11/20 cases overall (55%), or 11/18 successful policy episodes (61.1%). Mean reward regret was 0.663 among successful policy episodes and 2.209 when failures were included. This result shows that the policy often finds an optimal-reward route in these small, static instances; it does not establish optimality on the full dynamic network, and reward optimality is distinct from minimizing hops or maximizing delivered key bits.

6. DISCUSSION AND LIMITATIONS

A. Key-Store Utilization

The environment carries a normalized per-edge pool state and applies modeled depletion and replenishment terms during an episode. It does not represent a 10,000-key initialized buffer or refill from an absolute measured key rate. Since the environment has no multi-demand trace, its pool state cannot support comparable network-wide utilization or load-balancing results.

B. Management Traffic

The current simulator does not count controller queries, telemetry bytes, or control messages. Management traffic is not measured because neither a central-controller protocol nor local-observation exchange rules are implemented.

TABLE IV

PAIRED CROSS-PAIR AND SEASON/TIME ROUTE OUTCOMES FOR ONE TRAINED CHECKPOINT. HOP MEANS ARE CONDITIONAL ON SUCCESSFUL EPISODES.

Policy	Successes / episodes	Success rate	Wilson 95% CI	Mean successful hops	Invalid / fallback actions	No-valid-action states
GNN-PPO	99/108	91.7%	84.9–95.6%	31.94	0 / 0	0
BFS-hop	108/108	100.0%	96.6–100.0%	15.40	0 / 0	0
Dijkstra-km	33/108	30.6%	22.7–39.8%	96.36	0 / 0	0
Random	1/108	0.9%	0.2–5.1%	111.00	0 / 0	42
Max-SKR	0/108	0.0%	0.0–3.4%	–	0 / 0	10

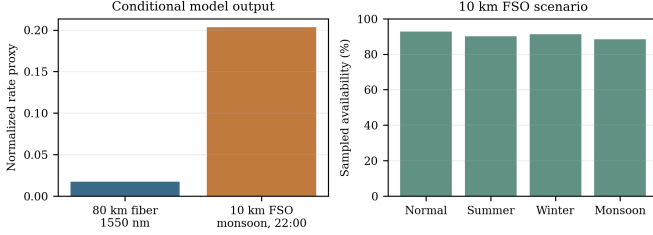


Fig. 8. Selected stored model outputs for fiber and FSO scenarios. Values are derived from the configured simulator, not measured traces.

1. Link Model Characterization

The stored physics report estimates, at 80 km and 1550 nm, 16.46 dB fiber loss including modeled excess loss, a detection probability of approximately 0.021 per incident photon, QBER proxy of 0.0105, and normalized rate proxy of 0.0175. This calculation uses an idealized one-photon-per-pulse count interpretation for a separate count-rate bound; weak-pulse source statistics are not included in that bound. For a 10-km FSO span during the monsoon 22:00 scenario, the modeled availability was 88% by configuration and 88.5% across 1,000 generated samples. Its median normalized rate proxy conditional on sampled available states was 0.2035. These values are generated by assumptions and should not be described as measured link performance.

2. Controlled Link-Choice Diagnostic

An earlier paired-edge diagnostic compared an 80-km fiber candidate with a clear 10-km FSO candidate while controlling route progress. In that archived experiment the policy failed to select the higher-proxy FSO candidate, motivating controlled feature-response tests. A later counterfactual analysis used a fixed shared route context and varied one link input at a time. The FSO action probability increased from 0.311 to 0.638 as the FSO-to-fiber proxy-rate ratio increased from 0.25 to 4.0. Within feasible QBER values, the probability changed only slightly (approximately 0.473 to 0.472); at QBER 0.111 the FSO candidate was correctly masked. Under a separate turbulence-input sweep, a 100-fold C_n^2 multiplier reduced the median FSO rate proxy from 0.202 to 0.183, while mean FSO action probability shifted only from 0.749 to 0.740 and FSO was selected in all 32 trials. Thus the checkpoint responds to proxy-rate contrast and the hard feasibility mask, but shows weak sensitivity to this turbulence feature scaling. These are synthetic input counterfactuals, not evidence of calibrated weather response or end-to-end route benefit.

C. Scope of the Current Evaluation

The completed route matrix covers one trained GNN checkpoint across six ordered pairs and six selected season/time conditions, with paired policy seeds. Because behavior cloning randomized endpoints and only one checkpoint is represented, these episodes do not establish strict endpoint generalization or variation across independent training runs. The simulator does not compare absolute delivered key volume, blocking under traffic, control-plane load, or performance on calibrated channel traces. The results therefore characterize a software prototype under selected simulated conditions, not network service performance.

Validation Levels

Validation is separated into implementation correctness, analytical checks, parameter calibration, synthetic stress testing, and hardware-in-the-loop or field evaluation. Implementation checks cover attenuation, detection, QBER, finite-key bounds, temporal correlation, outage behavior, and path constraints. Analytical validation compares code outputs with limiting cases and independently calculated reference values. Calibration requires traceable device and channel data with uncertainty records. Synthetic testing holds out entire weather traces and applies parameter shifts. Field validation requires timestamped measurements and separate reporting of model error and policy error.

Further Evaluation

The completed experiments provide an initial cross-pair/condition matrix, a paired comparison against four routing baselines, and an exact oracle on small static graphs. Further evaluation should use endpoint and weather traces withheld from both PPO and behavior cloning; multiple independently trained policies; matched-budget feature, architecture, and reward ablations; and absolute key generation, consumption, and refill under simultaneous demand. Report route success and blocking with uncertainty, successful route length, QBER feasibility, revisits, delivered key bits when calibrated, and inference cost. Hyperparameter selection and final test seeds must be separated.

Limitations

Current road routes are proxies and may not correspond to deployable fiber. FSO visibility and optical geometry are not validated. Weather distributions and effective turbulence scaling are assumptions. The fiber and FSO hardware settings do not represent one fully characterized terminal pair. The default rate expression is a normalized utility, and the finite-key implementation is not independently verified as a composable proof. The simulator supports algorithm development, but

this evidence does not establish field performance or certified security.

7. CONCLUSION

This paper has presented a graph-based reinforcement-learning simulator for routing in a modeled hybrid fiber and FSO trusted-node QKD network. In a paired 108-episode cross-pair and season/time matrix, one GNN checkpoint completed 91.7% of routes and substantially outperformed distance-weighted Dijkstra and greedy Max-SKR in completion rate, while making no invalid action proposals or triggering fallbacks. The exact small-graph study also found the GNN at the reward optimum in 61.1% of successful policy episodes. These results illustrate where the agent is useful in this simulator: it can use state-dependent feasibility and route context where fixed link-cost rules perform poorly.

The same evidence sets a clear limit. BFS completed all matrix episodes and used fewer hops than the GNN. We therefore make no claim that learned routing dominates the simplest shortest-hop baseline, generalizes to unseen endpoint-training distributions, maximizes delivered key bits, or improves network service. Such claims require strict holdouts, independent training replications, matched-budget ablations, absolute key-rate and demand models, and validated channel traces. Formal QKD security claims require protocol-specific finite-key analysis and independently checked count data.

ACKNOWLEDGMENT

Funding and institutional acknowledgments are to be supplied by the authors.

DATA AND CODE AVAILABILITY

The simulator code, archived configurations, scenario outputs, and generated figures are included with the supplementary research package. The road-corridor geometries derive from OpenStreetMap data and are provided under the Open Database License. Publications using the map should credit “Map data from OpenStreetMap contributors” and link to <https://www.openstreetmap.org/copyright>. The geometries are routing proxies and do not represent verified fiber infrastructure.

REFERENCES

- [1] P. W. Shor, “Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer,” *SIAM Journal on Computing*, vol. 26, no. 5, pp. 1484–1509, 1997.
- [2] V. Scarani, H. Bechmann-Pasquinucci, N. J. Cerf, M. Dušek, N. Lütkenhaus, and M. Peev, “The security of practical quantum key distribution,” *Reviews of Modern Physics*, vol. 81, no. 3, pp. 1301–1350, 2009.
- [3] M. Mehic, M. Niemiec, S. Rass, J. Ma, M. Peev, A. Aguado, V. Martin, S. Schauer, A. Poppe, C. Pacher, and M. Voznak, “Quantum key distribution: A networking perspective,” *ACM Computing Surveys*, vol. 53, no. 5, pp. 101:1–101:41, 2020.
- [4] T. Johann, M. Wenning, D. Giemsa, A. Dochhan, M. Gunkel, T. Feilenberger, and S. Pachnicke, “Comparison and optimization of different routing methods for meshed qkd networks using trusted nodes,” *Journal of Optical Communications and Networking*, vol. 16, no. 3, pp. 382–391, 2024.
- [5] W. J. Munro, K. Azuma, K. Tamaki, and K. Nemoto, “Inside quantum repeaters,” *IEEE Journal of Selected Topics in Quantum Electronics*, vol. 21, no. 3, p. 6400813, 2015.
- [6] “ITU-T Recommendation Y.3800: Overview on networks supporting quantum key distribution,” International Telecommunication Union, Geneva, Switzerland, 2019.
- [7] W. O. Krawec, B. Wang, and R. Brown, “Finite key security of simplified trusted node networks,” in *Proceedings of the IEEE International Conference on Quantum Computing and Engineering*, 2024, pp. 1777–1787.
- [8] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [9] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *Proceedings of the International Conference on Learning Representations*, 2017.
- [10] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph attention networks,” in *Proceedings of the International Conference on Learning Representations*, 2018.
- [11] C. H. Bennett and G. Brassard, “Quantum cryptography: Public key distribution and coin tossing,” in *Proceedings of the IEEE International Conference on Computers, Systems and Signal Processing*, Bangalore, India, 1984, pp. 175–179.
- [12] W.-Y. Hwang, “Quantum key distribution with high loss: Toward global secure communication,” *Physical Review Letters*, vol. 91, no. 5, p. 057901, 2003.
- [13] H.-K. Lo, X. Ma, and K. Chen, “Decoy state quantum key distribution,” *Physical Review Letters*, vol. 94, no. 23, p. 230504, 2005.
- [14] D. Gottesman, H.-K. Lo, N. Lütkenhaus, and J. Preskill, “Security of quantum key distribution with imperfect devices,” *Quantum Information and Computation*, vol. 4, no. 5, pp. 325–360, 2004.
- [15] M. Tomamichel, C. C. W. Lim, N. Gisin, and R. Renner, “Tight finite-key analysis for quantum cryptography,” *Nature Communications*, vol. 3, p. 634, 2012.
- [16] C. C. W. Lim, M. Curty, N. Walenta, F. Xu, and H. Zbinden, “Concise security bounds for practical decoy-state quantum key distribution,” *Physical Review A*, vol. 89, no. 2, p. 022307, 2014.
- [17] “ITU-T Recommendation Y.3803: Key management for quantum key distribution networks,” International Telecommunication Union, Geneva, Switzerland, 2020.
- [18] *ITU-R Recommendation P.1814-1: Prediction Methods Required for the Design of Terrestrial Free-Space Optical Links*, International Telecommunication Union Std., 2025.
- [19] A. Kržič, S. Sharma, C. Spiess, U. Chandrashekara, S. Töpfer, G. Sauer, L. J. G.-M. del Campo, T. Kopf, S. Petschornig, T. Grafenauer, R. Lieger, B. Ömer, C. Pacher, R. Berlich, T. Peschel, C. Damm, S. Risse, M. Goy, D. Rieländer, A. Tünnermann, and F. Steinlechner, “Towards metropolitan free-space quantum networks,” *npj Quantum Information*, vol. 9, p. 95, 2023.
- [20] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.
- [21] J. Roik, K. Bartkiewicz, A. Černoch, and K. Lemr, “Routing in quantum communication networks using reinforcement machine learning,” *Quantum Information Processing*, vol. 23, no. 89, 2024.
- [22] O. Amer, W. O. Krawec, V. U. Manfredi, and B. Wang, “Dynamic routing for quantum key distribution networks,” *arXiv preprint arXiv:2212.03144*, 2023.
- [23] R. Bellman, *Dynamic Programming*. Princeton, NJ, USA: Princeton University Press, 1957.
- [24] J. Schulman, P. Moritz, S. Levine, M. I. Jordan, and P. Abbeel, “High-dimensional continuous control using generalized advantage estimation,” in *Proceedings of the International Conference on Learning Representations*, 2016.
- [25] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra, “Planning and acting in partially observable stochastic domains,” *Artificial Intelligence*, vol. 101, no. 1–2, pp. 99–134, 1998.
- [26] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.
- [27] W. Hamilton, Z. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 1024–1034.
- [28] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, “Neural message passing for quantum chemistry,” in *Proceedings of the 34th International Conference on Machine Learning*, 2017, pp. 1263–1272.
- [29] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 5998–6008.
- [30] M. Haklay and P. Weber, “Openstreetmap: User-generated street maps,” *IEEE Pervasive Computing*, vol. 7, no. 4, pp. 12–18, 2008.